



Sede Regional Brunca

Campus Pérez Zeledón

Curso: Estructuras de Datos

Proyecto 1: Neon Tetris

Estudiante:

Luis Andres Elizondo Hernandez

Profesores: Saray María Castro Mora

Pablo Andrés Venegas Elizondo

II Ciclo - NRC 51005 - Grupo 82

27 de septiembre del 2026

Introducción y Descripción del Sistema

En este primer proyecto del curso de Estructuras de Datos desarrollamos una versión funcional de Tetris llamada "Neon Tetris" en C++, utilizando la biblioteca gráfica Raylib y el entorno Zinjal. El objetivo principal fue programar todas las estructuras de datos lineales vistas en clase (colas, pilas, listas enlazadas y ordenamientos) desde cero con nodos y punteros en memoria dinámica, sin usar las librerías estándar de C++ (STL).

El juego se desarrolla en un tablero de 10 columnas por 20 filas donde caen 7 tipos de piezas clásicas. Cada mecánica clave del juego fue pensada para resolverse con la estructura lineal que le corresponde de forma natural: la secuencia de piezas que van a salir usa una cola; la reserva de una pieza para después (hold) usa una pila; el historial para deshacer jugadas y el replay usa una lista doblemente enlazada; los eventos con temporizador usan una cola de prioridad; y el tablero completo se modela como una lista enlazada de filas. Además, implementamos dos métodos de ordenamiento (Burbuja y Merge Sort) para ordenar la tabla de puntajes guardada en JSON y comparamos sus tiempos de ejecución reales para ver cómo rinden en la práctica.

Arquitectura del Sistema y Relación entre Estructuras

El proyecto está dividido en tres carpetas para mantener el código ordenado: estructuras (clases de datos y punteros), lógica (piezas, rotaciones precalculadas y ordenamientos) y ui (pantallas y diseño visual con Raylib). En el archivo principal main.cpp se conectan todas las estructuras en un bucle que funciona de la siguiente manera:

- **Cola de piezas siguientes (Cola):** Abastece las piezas del juego. Para evitar repeticiones excesivas, usamos el sistema oficial de bolsa de 7 mezcladas con Fisher-Yates en $O(n)$. Conforme caen se extraen con desencolar() en $O(1)$, y en pantalla se previsualizan las 3 siguientes consultando con obtenerEn(k). Al vaciarse la cola, se baraja y encola una nueva bolsa automáticamente.
- **Pila para la pieza en espera (Pila):** Permite apartar la pieza actual para usarla después con Shift Derecho. Se modeló con una pila de capacidad 1. Si está vacía, la pieza actual se apila y se extrae una nueva de la cola; si ya había una guardada, se intercambian. Una bandera asegura un solo cambio por turno hasta que la pieza caiga.
- **Tablero como lista de filas (Tablero):** El tablero es una lista enlazada de 20 filas por 10 columnas donde cada celda apunta a sus vecinas (arriba, abajo, izquierda, derecha). Al completarse filas, tras una animación de destello de 250 ms, limpiarLineas() desconecta los

punteros verticales de las filas llenas, libera sus nodos con delete e inserta filas vacías arriba, haciendo que las filas superiores desciendan de forma natural por enlace.

- **Lista doblemente enlazada para historial y replay (ListaDoble):** Al fijar una pieza o limpiar líneas, se almacena una copia del tablero y puntaje en un nodo de la lista doble. Sus punteros siguiente y anterior permiten deshacer (Z) o rehacer (Y) jugadas en vivo, y al finalizar la partida alimentan el modo Replay paso a paso o en reproducción automática cada 0.5 s.

- **Cola de prioridad para eventos programados (ColaPrioridad):** Planifica eventos dinámicos temporizados (aumento de velocidad, líneas de basura por Terremoto o bonos). La cola mantiene los eventos ordenados por tiempo: el evento más próximo queda al frente en la posición 0 (consulta en $O(1)$), y al cumplirse el tiempo se extrae en $O(\log n)$ y se aplica su efecto.

- **Persistencia y ordenamiento de puntajes:** Al terminar la partida, el puntaje se registra en scores.json con ManejadorJSON. En la pantalla de Ranking se puede alternar entre Bubble Sort ($O(n^2)$) y Merge Sort ($O(n \log n)$), visualizando en pantalla el tiempo exacto de ordenamiento en microsegundos.

Complejidad Teórica Asintótica (Notación Big-O)

En la siguiente tabla resumimos la complejidad teórica de las operaciones principales de las estructuras implementadas en el proyecto:

Estructura	Operación	Mejor	Promedio	Peor	Espacio	Descripción Breve
Cola FIFO	encolar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Inserta directo al final usando el puntero ultimo.
Cola FIFO	desencolar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Saca del frente y libera el nodo con delete.
Cola FIFO	obtenerEn(k)	$O(1)$	$O(k)$	$O(n)$	$O(1)$	Recorre hasta la posición k para ver piezas futuras.
Pila LIFO	apilar() / desapilar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Inserta o extrae directamente en el tope.
Lista Doble	agregarEstado()	$O(1)$	$O(1)$	$O(1)$	$O(F \cdot C)$	Inserta al final; clona la matriz de 20x10 y puntaje.
Lista Doble	retroceder() [Undo]	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Paso al nodo anterior: actual = actual->anterior.
Lista Doble	avanzar() [Redo]	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Paso al nodo siguiente: actual = actual->siguiente.
Cola Prioridad	encolar()	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Inserta y se acomoda comparando con anteriores.
Cola Prioridad	desencolar()	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Saca evento del frente y reacomoda el arreglo.
Cola Prioridad	verTiempoFrente()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Consulta directa al primer evento en arreglo[0].
Tablero	limpiarLineas()	$O(F \cdot C)$	$O(F \cdot C)$	$O(F \cdot C)$	$O(1)$	Reconecta punteros verticales y crea filas nuevas arriba.
Ordenamiento	Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Comparaciones e intercambios adyacentes de vecinos.
Ordenamiento	Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Divide recursivamente y mezcla sub-arreglos.

4. Comparación Empírica y Benchmark (Burbuja vs. Merge Sort)

Medimos con <chrono> en C++ el tiempo real en microsegundos ordenando listas aleatorias de jugadores con puntajes entre 0 y 100 000 para N = 10, 100, 1 000 y 10 000 registros (promedio de 5 corridas por caso):

Tamaño (N)	Bubble Sort (µs)	Bubble Sort (ms)	Merge Sort (µs)	Merge Sort (ms)	Diferencia de Rendimiento
N = 10	0.90 µs	0.0009 ms	1.00 µs	0.0010 ms	Empate técnico (0.9x)
N = 100	117.50 µs	0.1175 ms	38.50 µs	0.0385 ms	Merge 3.05x más rápido
N = 1 000	12 333.00 µs	12.33 ms	650.00 µs	0.65 ms	Merge 18.97x más rápido
N = 10 000	1 309 150.00 µs	1 309.15 ms	5 857.00 µs	5.86 ms	Merge 223.51x más rápido

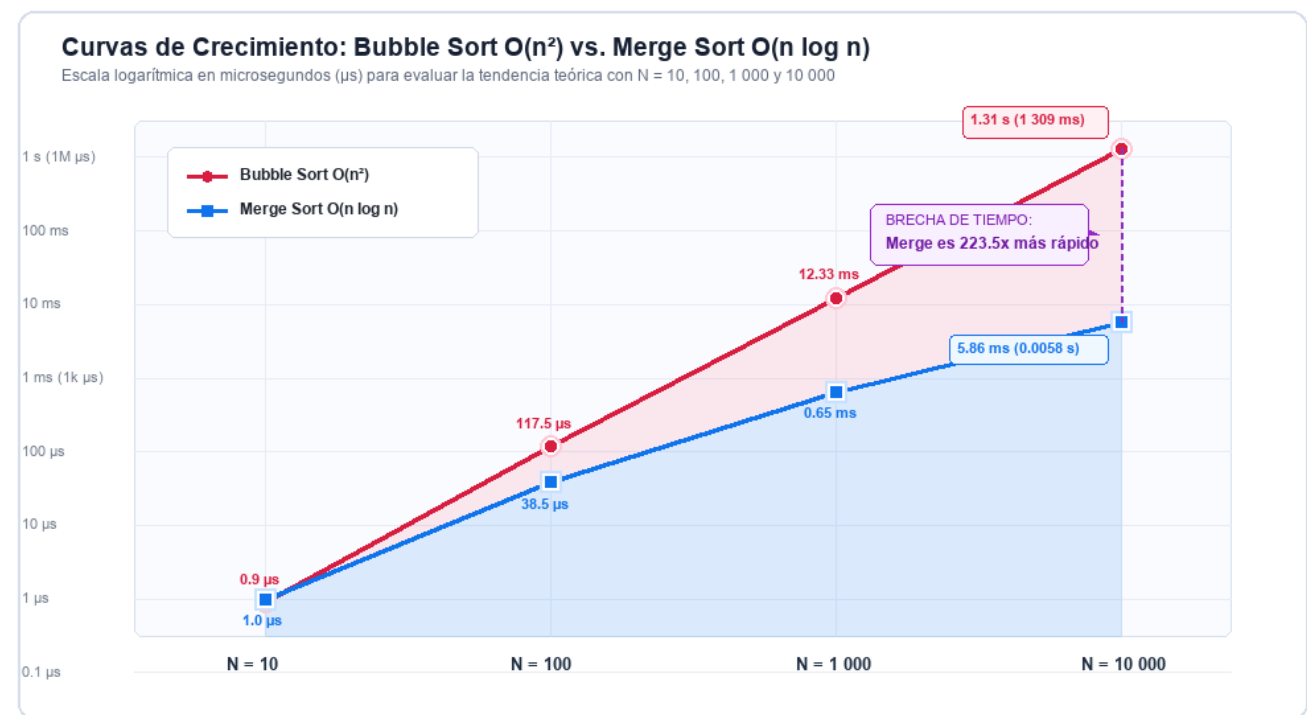


Figura 1: Curva de crecimiento en escala logarítmica comparando Bubble Sort y Merge Sort.

Análisis e Interpretación de Resultados:

Caso / Tamaño (N)	Resultado Observado	Interpretación y Causa Técnica
N = 10	Empate técnico (0.90 µs vs 1.00 µs)	Bubble Sort es una fracción más rápido porque opera directamente sobre el arreglo sin el sobrecosto de llamadas recursivas ni reserva de memoria para la mezcla.
N = 100	Merge Sort 3.05x más rápido (38.50 µs frente a 117.50 µs)	Se supera el umbral de eficiencia donde el enfoque recursivo divide y vencerás sobrepasa a las comparaciones cuadráticas adyacentes.
N = 1 000	Merge Sort 18.97x más rápido (0.65 ms frente a 12.33 ms)	La brecha es clara en la fluidez del juego: Bubble Sort toma más de 12 ms, mientras que Merge Sort resuelve el ordenamiento de forma instantánea.
N = 10 000	Merge Sort 223.51x más rápido (5.86 ms vs 1 309.15 ms)	Bubble Sort supera los 1.3 segundos (provocaría congelamiento de pantalla perceptible), mientras Merge Sort procesa todo en menos de 6 ms a 60 FPS estables.
Crecimiento (1k → 10k)	Bubble aumentó 106x Merge aumentó solo 9x	Confirma la teoría: al multiplicar datos por 10, Bubble Sort escala de forma cuadrática $O(n^2)$, mientras Merge Sort muestra un crecimiento casi lineal $O(n \log n)$.

5. Respuestas a las preguntas de análisis

Pregunta 1: Comparación de $O(n^2)$ vs. $O(n \log n)$ y umbral de diferencia

Con pocos datos ($N = 10$), ambos algoritmos empatan técnicamente en rendimiento ($0.9 \mu s$ para Bubble Sort y $1.0 \mu s$ para Merge Sort). Para tamaños muy pequeños las pocas operaciones directas de Bubble Sort son tan rápidas que compensan no tener la complejidad de Merge Sort, el cual gasta tiempo extra llamando funciones recursivas y pidiendo memoria para su arreglo temporal. La diferencia comienza a notarse claramente a partir de $N = 100$, donde Merge Sort ya es 3 veces más rápido. Con $N = 1\,000$ ya es 19 veces más veloz, y con $N = 10\,000$ la brecha es total: Bubble Sort tarda 1.31 segundos y Merge Sort apenas 5.86 milisegundos (más de 223 veces más rápido).

Sí coincide plenamente con lo esperado: Bubble Sort es cuadrático $O(n^2)$ multiplicando su tiempo por 100 al multiplicar datos por 10 (en nuestra prueba aumentó 106 veces), mientras que Merge Sort escala como $O(n \log n)$ con un crecimiento mucho más suave y eficiente.

Pregunta 2: Representación del tablero como lista enlazada de filas y costo de operaciones

En un arreglo estático común de 20×10 , al llenarse una fila en el medio se requieren bucles para copiar y mover físicamente hacia abajo todas las celdas superiores (hasta 190 copias de memoria innecesarias). Al modelar el tablero como una lista enlazada de filas, las filas superiores no se mueven en la memoria: simplemente se desconectan los enlaces verticales de la fila completada, se reconecta la fila inmediatamente superior con la inferior y se libera la fila eliminada. Las filas superiores bajan de forma natural porque sus enlaces ahora apuntan hacia abajo a la fila correcta sin tocar su contenido interno.

- Costo de eliminar fila: Se recorren las 10 columnas desconectando punteros y liberando celdas con delete. Toma 10 liberaciones y 20 reconexiones: costo $O(C)$ operaciones ($O(1)$ constante con $C = 10$).
- Costo de insertar fila vacía: Se crean 10 celdas vacías con new, se enlazan horizontalmente y se conectan sus punteros abajo con la fila superior existente. Toma 10 creaciones y 30 asignaciones de puntero: costo $O(C)$ ($O(1)$ constante).

Pregunta 3: Necesidad de la lista doblemente enlazada para el replay y costo de avanzar y retroceder

Para implementar deshacer y el replay requerimos movernos en el tiempo en ambas direcciones. En una lista simplemente enlazada cada nodo solo conoce al nodo siguiente; avanzar es directo (actual = actual->siguiente), pero retroceder un solo paso obligaría a

buscar desde la cabeza de la lista nodo por nodo hasta encontrar cuál apuntaba al actual. Con k jugadas, retroceder tomaría $O(k)$ operaciones, provocando congelamientos en pantalla. La lista doblemente enlazada resuelve esto almacenando punteros siguiente y anterior en cada estado:

- Avanzar un paso (`historial.avanzar()` / Tecla Y o botón Replay Adelante): Solo hace `actual = actual->siguiente` en $O(1)$ tiempo constante.
- Retroceder un paso (`historial.retroceder()` / Tecla Z o botón Replay Atrás): Solo hace `actual = actual->anterior` en $O(1)$ tiempo constante.

Pregunta 4: Garantía del evento más próximo al frente y costo de inserción en la cola de eventos

La cola de eventos mantiene los elementos ordenados por tiempo. Al insertar un nuevo evento, este se ubica y compara con los elementos anteriores para acomodarse en su posición exacta según el segundo en que debe dispararse. Así, el evento con menor tiempo (el que debe ocurrir antes) queda siempre en la primera posición (`arreglo[0]`). Para verificar si corresponde activarlo, el juego solo consulta este primer elemento en $O(1)$ tiempo constante. Insertar un evento nuevo toma $O(\log n)$ operaciones porque se divide el espacio a la mitad en cada paso, evitando reordenar toda la estructura desde cero.